

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 12

HTML JavaScript APIs

12.1 Allgemein

Die Vorlesungen 1 und 2 gaben eine Einführung in HTML, wobei immer von seiner letztgültigen Version – dem Living Standard – ausgegangen wurde. Themen waren hierbei vor allem semantische und strukturgebende Elemente, wie etwa `<header>`, `<footer>`, `<nav>` oder `<main>`, genauso aber auch neue Formularfeld-Typen wie etwa `<input type="email">` oder `<input type="number">`.

Dies ist jedoch bei weitem nicht alles, was HTML zum Zeitpunkt seiner Einführung an Neuem mitbrachte. Dinge wie eine eigene Zeichenfläche, die Möglichkeit, Video ohne Plugin abzuspielen oder auch die Fähigkeit, die Position von Benutzer*innen abzufragen, sind mächtige Werkzeuge. Während HTML zwar dafür eigene Elemente zur Verfügung stellt, ist der Zugriff auf diese Möglichkeiten jedoch nur mit JavaScript möglich, weshalb dieses Themengebiet erst jetzt angeschnitten wird.

Genau genommen, gehören einige dieser Funktionalitäten gar nicht zur HTML-Spezifikation. Technologien wie Canvas, Video, Web Storage sowie Drag & Drop sind zwar Bestandteile des HTML Living Standards [5], Geolocation [9] und auch andere (siehe Abschnitt 12.8) sind eigene Spezifikationen, die jedoch meist unter dem Oberbegriff HTML mitverwendet werden. Man spricht in diesem Zusammenhang von HTML JavaScript APIs oder auch einfach Web APIs (API = Application Programming Interface, deutsch „Programmierschnittstelle“), da ein programmatischer Zugriff auf die Daten in den HTML-Elementen ermöglicht wird.

12.1.1 HTML-APIs und Feature Detection

Moderne Browser unterstützen die meisten der hier gezeigten APIs. Dennoch ist es gute Praxis, zu überprüfen, ob ein Feature verfügbar ist, bevor es genutzt wird. Dies nennt man *Feature Detection*.

Entwickler*innen können dazu bei *Can I use...*¹ nachschlagen und feststellen, ab welcher Browserversion bestimmte Funktionalitäten verfügbar sind.

Um programmatisch (d. h. im JavaScript-Code) die aktuellen verfügbaren Fähig-

¹<https://caniuse.com/>

keiten eines Browsers festzustellen, wurden früher eigene Bibliotheken wie *Modernizr*² eingesetzt. Heute geschieht dies meist direkt mit JavaScript. Um etwa zu prüfen, ob Geolocation verfügbar ist, reicht eine einfache If-Abfrage mit dem `in`-Operator, der `true` zurückgibt, wenn eine Eigenschaft (hier `geolocation`) in einem Objekt (hier `navigator`) vorhanden ist:

```
1 if ("geolocation" in navigator) {
2   // Geolocation ist verfügbar
3 } else {
4   // Fallback oder Fehlermeldung
5 }
```

Wurde festgestellt, dass eine Funktionalität fehlt, kann in seltenen Fällen ein sogenannter *Polyfill* (ein Nachbau der Funktion in JavaScript) nachgeladen werden.

Feature Detection für CSS

Oft ist es notwendig, nicht nur JavaScript-APIs, sondern auch die Unterstützung für bestimmte CSS-Eigenschaften zu prüfen (z. B. für komplexe Layouts wie `subgrid` oder grafische Effekte). Während dies im Stylesheet direkt mit der `@supports`-Regel³ geschehen kann, ist es manchmal notwendig, diese Information im JavaScript-Code zu haben.

Die moderne Methode hierfür ist die Funktion `CSS.supports()`⁴. Sie gibt `true` zurück, wenn der Browser die Kombination aus Eigenschaft und Wert bzw. die Bedingung versteht:

```
1 // Prüfen, ob eine Eigenschaft mit einem Wert existiert
2 if (CSS.supports("display", "grid")) {
3   // Grid-spezifische Logik aktivieren
4 }
5
6 // Eine Bedingung, wie bei @supports prüfen
7 if (CSS.supports("selector(&)")) {
8   console.log("CSS Nesting wird unterstützt");
9 }
```

12.2 Canvas

Die Definition des `<canvas>`-Elements ist in [5, Abschnitt 4.12.5] wie folgt festgelegt:

The canvas element provides scripts with a resolution-dependent bitmap canvas, which can be used for rendering graphs, game graphics, art, or other visual images on the fly.

Es handelt sich also um eine „auflösungsabhängige Bitmap-Zeichenfläche, die zur on-the-fly Darstellung von Graphen, Spielegrafiken, Kunst oder anderen Bildern“ verwendet werden kann.

²<https://modernizr.com/>

³<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@supports>

⁴https://developer.mozilla.org/en-US/docs/Web/API/CSS/supports_static

Ein `<canvas>`-Element wird zunächst in HTML mit einem `width` und `height` Attribut – also konkreten Dimensionen in Pixeln – angelegt. Zur leichteren Auffindung in JavaScript wird eine ID vergeben:

```
1 <canvas width="640" height="480" id="draw">Ihr Browser unterstützt Canvas leider  
   nicht!</canvas>
```

Dies erzeugt eine Zeichenfläche mit 640×480 Pixeln, jedoch ohne jeglichen Inhalt. Der Text zwischen `<canvas>` und `</canvas>` wird dabei nur angezeigt, falls der Browser das Element nicht unterstützt. Dies könnte aber auch über JavaScript zuvor überprüft und anders abgefangen werden, sollte JavaScript aber deaktiviert sein, ist ein derartiger „Fallback“-Text durchaus sinnvoll.

Um nun im *Canvas* „zeichnen“ zu können, muss der sogenannte *Zeichenkontext* (engl. „drawing context“) abgefragt werden. Er repräsentiert quasi den editierbaren Inhalt. Derzeit existiert ein im HTML Living Standard definierter 2D-Kontext [5, Abschnitt 4.12.5.1], die Technologie WebGL⁵ bietet jedoch auch einen 3D-Kontext [2, 3], der sich an der OpenGL ES-Spezifikation orientiert.

Alle Zugriffe auf das in HTML angelegte `<canvas>`-Element erfolgen nun in JavaScript. Im folgenden Code wird das Element gesucht und sein Kontext abgefragt:

```
1 const canvas = document.getElementById("draw");  
2 const ctx = canvas.getContext("2d");
```

Sobald der Kontext vom `<canvas>`-Element abgefragt wurde, kann auf verschiedene Arten Inhalt hinzugefügt werden.

12.2.1 Rechtecke

Als einfachste zu zeichnende Form bieten sich Rechtecke an. Mehr Grundformen (wie etwa Kreise) sind derzeit nicht vorhanden.

Mit Hilfe des abgefragten Context-Objekts können drei Methoden zum Zeichnen von Rechtecken verwendet werden:

- **fillRect(x, y, width, height)**: Zeichnet ein gefülltes Rechteck an der Position (x, y) mit der Breite *width* und Höhe *height* füllt es mit dem aktuellen Füll-Stil.
- **strokeRect(x, y, width, height)**: Zeichnet ein Rechteck ohne eine Füllung aber mit Rahmenlinie an der Position (x, y) mit der Breite *width* und Höhe *height*. Die Art der Linie wird vom aktuell eingestellten Linien-Stil vorgegeben.
- **clearRect(x, y, width, height)**: Löscht die Pixel, die vom Rechteck an der Position (x, y) mit der Breite *width* und Höhe *height* verdeckt würden.

Füll- und Linien-Stil

Wie ein gefülltes oder umrandetes Rechteck genau dargestellt wird, hängt vom Füll- bzw. Linien-Stil ab. Diese werden über die beiden Eigenschaften `fillStyle` und `strokeStyle` beim Kontext gesetzt. Als Werte können entweder CSS-Farbangaben wie `blue`, `red` oder `#00FF00` oder ebenfalls mit JavaScript erzeugte Farbverläufe (siehe [5, Abschnitt 4.12.5.1.10]) verwendet werden.

⁵<https://www.khronos.org/webgl/>

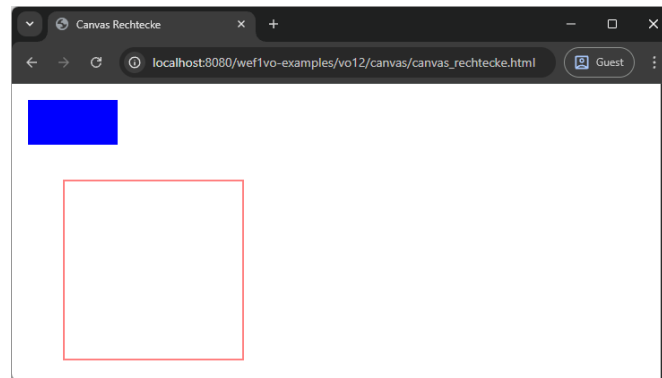


Abbildung 12.1: Durch die gesetzten Füll- und Linien-Stile sind ein blaues, gefülltes Rechteck mit 100×50 Pixeln und ein transparentes Rechteck mit roter Linie und 200×200 Pixeln entstanden. Die Ausmaße des Canvas selbst betragen 640×480 Pixel, was die Scrollbalken beim Browserfenster erklärt.

Werden der *Füll-* oder *Linienstil* gesetzt, gelten sie so lange, bis sie neu gesetzt werden, der Canvas zurückgesetzt oder die Seite neu geladen wird. Die Standardangabe ist jeweils schwarz.

Im folgenden Ausschnitt wird der Füll-Stil auf blau und der Linien-Stil auf rot gesetzt, danach werden zwei Rechtecke gezeichnet. Das Ergebnis ist in Abbildung 12.1 ersichtlich.

```
1 ctx.fillStyle = "blue";  
2 ctx.strokeStyle = "#FF0000";  
3 ctx.fillRect(10, 10, 100, 50);  
4 ctx.strokeRect(50, 100, 200, 200);
```

Mit der Eigenschaft `lineWidth` lässt sich darüber hinaus die Stärke einer Linie bestimmen. `lineCap` bestimmt, wie die Linie abgeschlossen wird. Mögliche Werte hierfür `butt`, `round` und `square`.

Koordinaten im Canvas

Wie im obigen Beispiel ersichtlich, wird das blaue Rechteck an den Koordinaten (10,10) positioniert. Beim Vergleich mit der Darstellung in Abbildung 12.1 wird schnell klar, dass sich der Ursprung des Koordinatensystems links oben befindet. Abbildung 12.2 stellt dies schematisch dar.

12.2.2 Pfade

Fortgeschrittenere Formen lassen sich über *Pfade* realisieren. Das Vorgehen dabei gleicht dem Zeichnen mit einer Füllfeder. Diese wird an einem Punkt aufgesetzt und eine Linie bis zu einem zweiten Punkt gezogen. Doch bevor der Pfad in Tinte erscheint, wird er zunächst mit Bleistift vorgezeichnet und erst abschließend mit der Füllfeder permanent gezeichnet.

Um quasi mit dem Bleistift die Linien vorzuzeichnen, existieren zwei Methoden beim Kontext-Objekt:

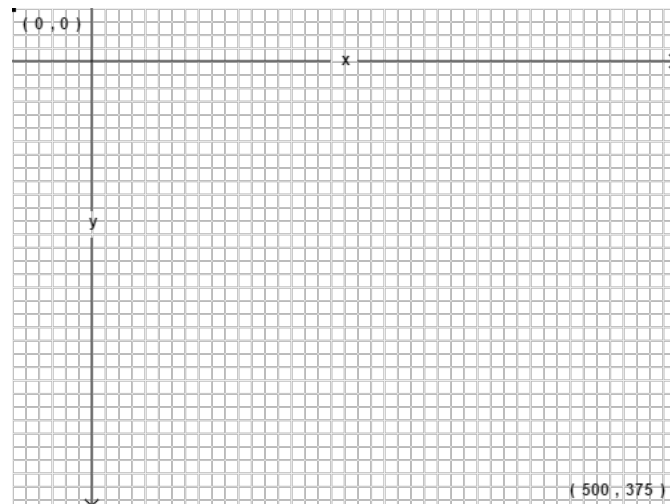


Abbildung 12.2: Der Ursprung des Canvas-Koordinatensystems liegt links oben. Die Abbildung ist aus [4, S. 60] entnommen und wurde selbst mit dem `<canvas>`-Element erzeugt.

- `moveTo(x, y)`: Setzt den Stift an die angegebene (x, y) -Position.
- `lineTo(x, y)`: Zieht eine Linie zur spezifizierten (x, y) -Endposition.

Diese Methoden lassen sich so oft wie gewünscht oder benötigt aufrufen, doch sie bewirken noch keine Darstellung der gezogenen Linien. Erst durch den Aufruf der Methode `stroke()` werden die Pfade gezeichnet, also quasi mit Tinte verewigt. Ist statt einer Linie eine Füllung des Pfades gewünscht, kann `fill()` aufgerufen werden.

Jeder Aufruf von `moveTo()` (also quasi jedes Neu-Aufsetzen des Stifts) erzeugt dabei einen eigenen Sub-Pfad. Soll dieser schnell geschlossen werden, also eine Linien vom zuletzt erstellten zum ersten Punkt gezogen werden, kann `closePath()` aufgerufen werden.

Die Methode `beginPath()` entfernt alle bisherigen erstellten und nicht mit `stroke()` oder `fill()` gezeichneten Sub-Pfade und erlaubt es so, weitere Pfade zu zeichnen, die nicht mit alten zusammenhängen.

Schließlich können auch andere Formen außer einfache Linien erstellt werden:

- `arc(x, y, radius, startAngle, endAngle)`: Erstellt einen ganzen Kreis (bzw. Kreisteil), mit Mittelpunkt (x, y) , dem Radius *radius* und gegebenen Anfangs- und Endwinkeln.
- `rect(x, y, width, height)`: Erstellt einen geschlossenen Pfad, der ein Rechteck repräsentiert.
- `quadraticCurveTo(cpx, cpy, x, y)`: Erstellt vom aktuellen Punkt aus eine quadratische Bézierkurve zum Endpunkt (x, y) mit dem Kontrollpunkt (cpx, cpy) .
- `bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x, y)`: Erstellt ausgehend vom aktuellen Punkt eine kubische Bézierkurve zum Endpunkt (x, y) mit den Kontrollpunkten $(cp1x, cp1y)$ und $(cp2x, cp2y)$.
- `arcTo(x1, y1, x2, y2, radius)`: Erstellt vom aktuellen Punkt aus einen Kreis

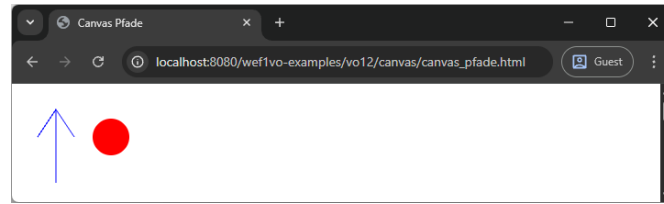


Abbildung 12.3: Zunächst wird die linke Seite der Pfeilspitze gezeichnet, dann die rechte und schließlich die Linie. Dann wird ein neuer Pfad begonnen und ein Kreis an der angegebenen Position erzeugt.

zum Endpunkt (x, y) mit den Kontrollpunkten (x_1, y_1) und (x_2, y_2) und dem Radius *radius*.

Der folgende Code für eine Verwendung im Canvas realisiert einen blauen Pfeil sowie einen rot gefüllten Kreis. Das Ergebnis ist in Abbildung 12.3 zu sehen.

```
1 ctx.strokeStyle = "blue";
2 ctx.moveTo(20, 50);
3 ctx.lineTo(40, 20);
4 ctx.lineTo(60, 50);
5 ctx.moveTo(40, 20);
6 ctx.lineTo(40, 100);
7 ctx.stroke();
8 ctx.beginPath(); // Beginne neuen unabhängigen Pfad
9 ctx.fillStyle = "red";
10 ctx.arc(100, 50, 20, 0, 2 * Math.PI)
11 ctx.fill();
```

12.2.3 Text

Text im Canvas kann mit Hilfe einiger weniger Angaben relativ schnell gesetzt werden. Dazu ist es zuerst nötig, die gewünschte Schrift anzugeben. Dies erfolgt über die Eigenschaft `font`. Diese akzeptiert eine CSS Schriften-Regel, genau wie die `font`-Eigenschaft in CSS (siehe dazu Abschnitt 3.8). Schriftgröße und -art müssen am Ende, andere Eigenschaften davor angegeben werden. Wird eine Schriftart nicht explizit angegeben, wird die für das `<canvas>`-Element geltende vererbt.

Um den Text dann darzustellen, können folgende zwei Methoden verwendet werden:

- `fillText(text, x, y)`: Schreibt den Text `text` an die Position (x, y) und füllt die Zeichen aus.
- `strokeText(text, x, y)`: Schreibt den Text `text` an die Position (x, y) , stellt jedoch nur die Outline dar.

Über die Eigenschaften `textAlign` (mit den Werten `start`, `end`, `left`, `right` und `center`) und `textBaseline` können die horizontale und vertikale Ausrichtung bestimmt werden.

Das folgende Beispiel setzt zwei Zeilen Text mit unterschiedlicher Ausrichtung. Abbildung 12.4 zeigt das Ergebnis.

```
1 ctx.font = "bold 24pt system-ui, sans-serif";
2 ctx.fillText("Hallo WEF1!", 200, 50);
```



Abbildung 12.4: Die x -Position der beiden Textzeilen ist gleich, die erste ist jedoch links-, die zweite rechtsbündig ausgerichtet. Einmal wird der Text gefüllt, beim anderen Mal wird nur die Outline dargestellt.

```
3 ctx.textAlign = "right";
4 ctx.strokeText("Hallo WEF1!", 200, 100);
```

12.2.4 Bilder

Um *Bilder* innerhalb eines Canvas darzustellen wird – wenig überraschend – ein Bild benötigt. Dieses kann entweder im HTML-Code bereits in Form eines `<img>`-Elements existieren oder auch rein in JavaScript angelegt werden.

Um das Bild im Canvas darzustellen, gibt es drei Methoden:

- `drawImage(image, dx, dy)`: Zeichnet das Bild `image` an die Zielkoordinaten (dx, dy) .
- `drawImage(image, dx, dy, dw, dh)`: Zeichnet das Bild `image` an die Zielkoordinaten (dx, dy) und skaliert es auf eine Breite von dw und eine Höhe von dh Pixeln.
- `drawImage(image, sx, sy, sw, sh, dx, dy, dw, dh)`: Entnimmt aus dem zu zeichnenden Bild `image` einen Ausschnitt mit der Breite sw und Höhe sh , ausgehend von (sx, sy) und zeichnet diesen an die Zielkoordinaten (dx, dy) und skaliert ihn auf eine Breite von dw und eine Höhe von dh Pixeln. Dies ist schematisch in Abbildung 12.5 dargestellt.

Existiert im HTML-Markup (d. h. im DOM-Baum) bereits ein `<img>`-Element und soll dessen Quelle als Bild verwendet werden, so muss es nur noch mit JavaScript abgefragt und einer `drawImage()`-Methode übergeben werden. Das Bild ist im HTML-Markup folgendermaßen definiert:

```
1 
```

Nun wird mittels JavaScript auf dieses Bild zugegriffen und es wird erneut im Canvas gezeichnet:

```
1 const img = document.getElementById("image");
2 ctx.drawImage(img, 20, 20);
```

Soll ein komplett neues Bild im Canvas geladen werden, so muss dafür in JavaScript zunächst ein `Image`-Objekt angelegt werden. Dieses repräsentiert ein Bild genauso, wie es durch das `<img>`-Element in HTML repräsentiert wird. Die Bilddatei wird dem Objekt im `src`-Attribut spezifiziert. Wichtig ist nun aber, dass die Darstellung im Canvas erst erfolgen darf, wenn das Bild vollständig geladen ist, daher muss der Aufruf von

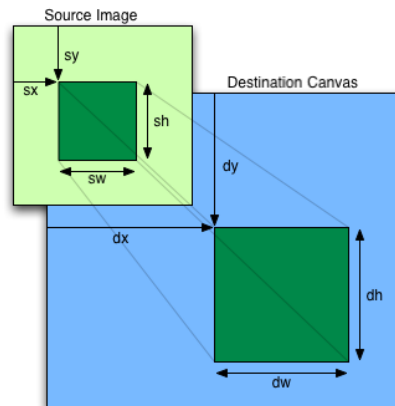


Abbildung 12.5: Aus dem Quellbild (hellgrün) wird ausgehend von der Position (sx, sy) ein Rechteck mit Breite sw und Höhe sh entnommen und in den Canvas (blau) an der Stelle (dx, dy) eingefügt. Der Ausschnitt wird dabei auf die Breite dw und die Höhe dh skaliert. Bildquelle: [5, Abschnitt 4.12.5.1.15].

`drawImage()` innerhalb eines `load`-Event-Handlers beim `Image`-Objekt erfolgen. Der Code hierfür lautet wie folgt:

```
1 const jsImg = new Image();
2 jsImg.src = "../assets/html5logo.png";
3 jsImg.addEventListener("load", function () {
4   ctx.drawImage(jsImg, 0, 24, 92, 104, 150, 60, 46, 52);
5 });
```

Abbildung 12.6 zeigt das Ergebnis beider Verfahren, wobei bei der zweiten Möglichkeit nur ein Ausschnitt aus der Grafik entnommen und skaliert dargestellt wird.

Achtung: Da hier mit einer externen Ressource (dem Bild aus dem DOM) gearbeitet wird, sollte dieser Code auf jeden Fall in einen Event-Listener zum Warten auf das vollständige Laden der Seite verpackt werden. Dabei muss auf den `load`- und nicht auf `DOMContentLoaded`-Event gewartet werden, damit das Bild auch verfügbar ist, wenn in JavaScript darauf zugegriffen wird.

12.2.5 Beispiele

Es existieren bereits zahllose Beispiele zum Thema Canvas. Hier eine kleine Auswahl:

- <https://zenphoton.com/>: Zen Photon Garden von Micah Elizabeth Scott.
- <https://git.nihilogic.dk/wolf3d/>: JavaScript Wolfenstein 3D von Jacob Seidelin.
- <http://media.chikuyonok.ru/ambilight/>: Ambilight for Video Tag von Sergey Chikuyonok.
- <https://viliusle.github.io/miniPaint/>: miniPaint von ViliusL.

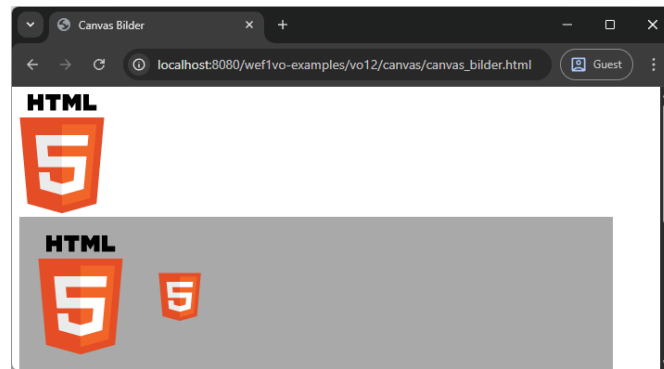


Abbildung 12.6: Die Grafik im weißen Bereich wurde in HTML mittels `<img>`-Element erzeugt. Der Canvas ist zur besseren Übersicht grau hinterlegt. Die linke Grafik ist die Kopie der in HTML erstellten, die rechte wurde komplett in JavaScript angelegt und dargestellt. Dabei kommt die Version von `drawImage()` zum Einsatz, die es erlaubt, einen Ausschnitt zu entnehmen und das Ergebnis zu skalieren.

12.3 Video

Video im Browser war lange Zeit nur durch Verwendung von Plugins wie Adobe Flash oder Apple QuickTime möglich. Trotz der großen Verbreitung dieser Plugins gab es dennoch Limitierungen, wie etwa schlechter Support auf mobilen Plattformen (bzw. völlig fehlender Support von Flash auf iOS). Es wurde daher in HTML das Element `<video>` (geschlossen durch `</video>`) definiert, um einen Standard zu schaffen, der in allen Browsern ohne zusätzliche Technologien funktioniert.

12.3.1 Container und Codecs

Beim Thema Videodateien fallen schnell Formate wie MP4 ein. Bei diesen handelt es sich aber genau genommen nur um sogenannte *Containerformate*. Container definieren nur, *wie* Inhalte gespeichert werden, nicht *welche* Art von Inhalten. Letzteres wird wiederum über sogenannte *Codecs* realisiert. Diese übernehmen die Dekodierung der Video- und Audiodaten und ermöglichen eine Ausgabe als Video am Bildschirm.

Während es im Desktopbereich eine Vielzahl von Containerformaten gibt, sind für HTML-Video nur zwei von Bedeutung:

- **MP4:** Kurz für MPEG-4. Bekannte Dateierendungen sind `.mp4` oder `.m4v`.
- **WebM:** Ein Open-Source Format von Google, die Dateierendung lautet `.webm`.

Folgende Video-Codecs sind werden von aktuellen Browsern unterstützt:

- **H.264:** Ein weit verbreiteter Video-Kompressionsstandard, der hohe Qualität bei relativ niedrigen Bitraten bietet und in vielen Anwendungen wie Streaming verwendet wird.
- **H.265:** Der Nachfolger von H.264, der etwa 25–50 % bessere Datenkompression bei gleicher Videoqualität ermöglicht und für hochauflösende Inhalte wie 4K und 8K optimiert ist.
- **VP8:** Ein offener und lizenzfreier Video-Codec, der von Google entwickelt wurde

Tabelle 12.1: Video-Containerformate mit passenden Video- und Audio-Codecs und Browserunterstützung. C144: Chrome 144, FF147: Firefox 147, E144: Edge 144, O125: Opera 125, S26.2: Safari 26.2.

<i>Format</i>	<i>C144</i>	<i>FF147</i>	<i>E144</i>	<i>O125</i>	<i>S26.2</i>
MP4: H.264 + AAC/MP3	•	○/•	•	•	•
MP4: H.265 + AAC	○	○	○	○	•
MP4: AV1 + AAC/Opus	•	○/•	•	•	•
WebM: VP8 + Vorbis	•	•	•	•	•
WebM: VP9 + Opus	•	•	•	•	•
WebM: AV1 + Opus	•	•	•	•	•

und als Alternative zu H.264 dient, aber inzwischen weitgehend durch neuere Formate ersetzt wurde.

- **VP9:** Der Nachfolger von VP8, ebenfalls von Google entwickelt, der eine verbesserte Kompressionseffizienz bietet und häufig für Streaming-Dienste wie YouTube verwendet wird.
- **AV1:** Ein offener, lizenzfreier Video-Codec, der darauf abzielt, bessere Kompression als H.265 und VP9 bei gleichzeitiger Vermeidung von Lizenzgebühren zu bieten.

Folgende Audio-Codecs können in Browsern derzeit eingesetzt werden:

- **MP3:** Ein weit verbreiteter verlustbehafteter Audiocodec, der gute Klangqualität bei relativ niedrigen Bitraten bietet und aufgrund seiner universellen Kompatibilität nach wie vor sehr populär ist.
- **AAC:** Ein fortschrittlicher verlustbehafteter Audiocodec, der als Nachfolger von MP3 entwickelt wurde und bei gleicher Bitrate eine bessere Klangqualität als MP3 erreicht.
- **Opus:** Ein vielseitiger Open-Source-Audiocodec mit sehr niedriger Latenz, der für interaktive Echtzeitanwendungen wie VoIP und Videokonferenzen optimiert ist und eine breite Palette von Bitraten und Audioqualitäten unterstützt.
- **Vorbis:** Ein freier, verlustbehafteter Audiocodec, der eine effiziente Kompression bei guter Klangqualität bietet, wobei er besonders für mittlere bis hohe Bitraten optimiert ist.

Aus dieser Auflistung an Containern, Video- und Audio-Codecs ist nun eine Reihe von Kombinationen möglich, die jeweils unterschiedlichen Browsersupport haben. Tabelle 12.1 stellt dies dar. Wie ersichtlich ist, ist die Unterstützung gut, lediglich Firefox kann MP4 mit AAC Audio Codec nur wiedergeben, wenn der Codec im Betriebssystem installiert ist und H.265 benötigt in allen Browsern (Ausnahme Safari) eine entsprechende Hardwareunterstützung. Die „sicheren“ Varianten sind derzeit H.264 Video mit MP3-Audio oder WebM, sicherheitshalber sollten jedoch mehrere Videoformate angeboten werden, um in allen Browsern ein Video angezeigt zu bekommen.

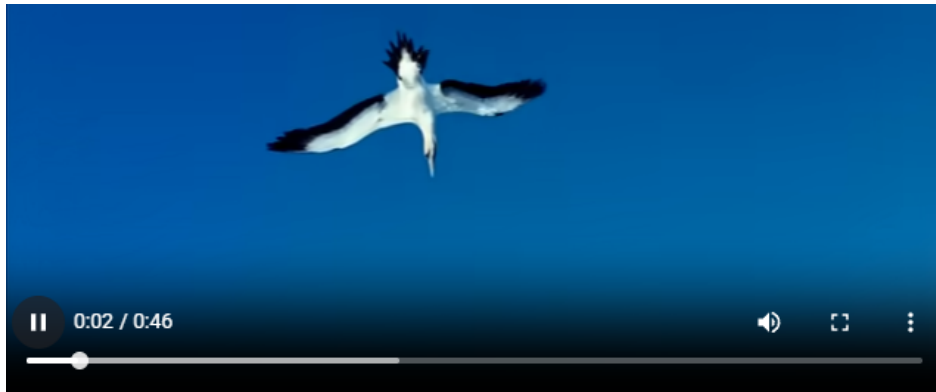


Abbildung 12.7: Chrome stellt das Video mit seinen eigenen Kontrollelementen dar.

12.3.2 Video Markup

Die einfachste Art und Weise, einen Videoclip mit dem `<video>`-Element wiederzugeben, ist die Angabe der Datei im `src`-Attribut, zusammen mit den Dimensionen des Videos (durch `width` und `height`). Durch die Angabe des Attributs `controls` lassen sich simple Kontrollelemente anzeigen. Text innerhalb des `<video>`-Elements wird nur dann angezeigt, wenn der Browser dieses Element nicht unterstützt.

```
1 <video src="../../../assets/oceans-clip_vp9_opus.webm" width="640" height="264" controls>
  Ihr Browser unterstützt Video leider nicht!</video>
```

Um nun aber sicher zu gehen, dass jeder Browser das Video wiedergeben kann, sollten beide verwendbaren Containerformate eingebunden werden. Da es aber nur ein `src`-Attribut beim `<video>`-Element gibt, muss hierbei anders vorgegangen werden. Das `<video>`-Element kann eine beliebige Anzahl von `<source>`-Elementen enthalten. Jedes dieser Elemente verweist in seinem eigenen `src`-Attribut auf die jeweilige Videodatei. Die Reihenfolge, in der die einzelnen Formate angegeben werden, ist freigestellt. Der Browser geht diese von oben nach unten durch und spielt die erste Datei, die er interpretieren kann, ab. Darüber hinaus müssen durch die Angabe des Attributs `type` der Container angegeben werden. Der folgende Code stellt eine Variante mit sechs Kombinationen aus der Tabelle dar, Abbildung 12.7 zeigt die Darstellung in Chrome.

```
1 <video width="640" height="264" controls>
2   <source src="../../../assets/oceans-clip_av1_opus.webm" type="video/webm">
3   <source src="../../../assets/oceans-clip_av1_opus.mp4" type="video/mp4">
4   <source src="../../../assets/oceans-clip_vp9_opus.webm" type="video/webm">
5   <source src="../../../assets/oceans-clip_h265_aac.mp4" type="video/mp4">
6   <source src="../../../assets/oceans-clip_h264_mp3.mp4" type="video/mp4">
7   <source src="../../../assets/oceans-clip_vp8_vorbis.webm" type="video/webm">
8 </video>
```

12.3.3 Kontrollelemente

Anstatt die vom Browser definierten *Kontrollelemente*, die durch die Angabe des HTML-Attributs `controls` zur Verfügung gestellt werden zu verwenden, kann das Video auch

über JavaScript gesteuert werden, d. h. es können eigene Kontrollelemente erstellt werden.

Im folgenden Beispiel werden ein Button für „Play“ und „Pause“ sowie zwei Buttons zur Veränderung der Lautstärke (+/−) erstellt. Diese werden über JavaScript angesprochen und steuern das Video. Das HTML-Markup für die Buttons sieht wie folgt aus, das Video ist auf die oben gezeigte Weise in dreifacher Ausführung eingebunden, das Video erhält weiters die ID `vid`.

```
1 <button type="button" id="playpause">Play</button>
2 <button type="button" id="volup">+</button>
3 <button type="button" id="voldown">-</button>
```

Nun werden in JavaScript die Buttons und das Video abgefragt:

```
1 const vid = document.getElementById("vid");
2 const playPause = document.getElementById("playpause");
3 const volUp = document.getElementById("volup");
4 const volDown = document.getElementById("voldown");
```

Nun wird auf jeden der drei Buttons ein `click`-Listener gelegt. Der Listener für den „Play/Pause“-Button fragt ab, ob das Video pausiert ist. Ist es pausiert, wird es abgespielt und umgekehrt. Je nach Status wird auch der Text des Buttons von „Play“ auf „Pause“ und umgekehrt geändert.

Bei den Buttons für die Lautstärkenregelung wird geprüft, ob die Lautstärke zwischen kleiner 1 und größer 0 ist, denn nur diese Werte sind möglich. Danach wird die Lautstärke um 0,1 nach oben oder unten verändert. Der Code dazu lautet wie folgt, Abbildung 12.8 stellt das Video mit den (einfachen) Kontrollelementen dar.

```
1 playPause.addEventListener("click", () => {
2   vid.paused ? vid.play() : vid.pause();
3   playPause.textContent = vid.paused ? "Play" : "Pause";
4 });
5 volUp.addEventListener("click", () => {
6   if (vid.volume < 1) {
7     vid.volume += 0.1;
8   }
9 });
10 volDown.addEventListener("click", () => {
11   if (vid.volume > 0) {
12     vid.volume -= 0.1;
13   }
14 });
```

Autoplay Policies und User Interaction

Moderne Browser verhindern das automatische Abspielen von Video- und auch Audiodateien mit Ton, um Nutzer*innen nicht zu belästigen. Ein Aufruf von `vid.play()` direkt beim Laden der Seite würde daher scheitern und in der Konsole einen Fehler (*NotAllowedError*) werfen.

Das hier gezeigte Beispiel funktioniert jedoch problemlos, da der Befehl `vid.play()` innerhalb eines **Click-Event-Listeners** ausgeführt wird. Der Browser erkennt, dass der Befehl als direkte Reaktion auf eine bewusste *User Interaction* (den Klick auf den Button) erfolgt, und erlaubt daraufhin die Wiedergabe. Möchte man ein Video zwingend automatisch ohne Interaktion starten (z. B. als Hintergrundvideo), muss es stummgeschaltet sein (Attribut `muted`).

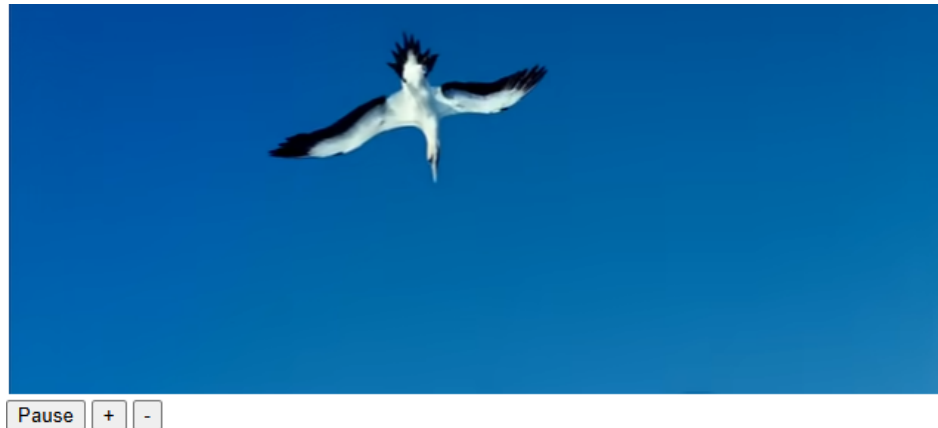


Abbildung 12.8: Das Video mit den selbst erstellten Kontrollelementen.

12.4 Audio

Das Abspielen und Steuern von reinen Audiodateien gestaltet sich sehr ähnlich dem von Video. Eingebunden werden Audiodateien über das `<audio>`-Element, geschlossen durch `</audio>`. Auch hier gibt es verschiedene Formate, die verwendet werden können.

12.4.1 Container und Codecs

Für den reinen Audiobereich sind dieselben Formate wie in Kombination mit Video verfügbar, darüber hinaus gibt es allerdings noch andere, die eingesetzt werden können. Wiederum gibt es eine Reihe von verschiedenen Container-Formaten:

- **MP3:** Ein Container für den gleichnamigen Codec, Endung lautet `.mp3`.
- **MP4:** Kurz für MPEG-4. Bekannte Dateierendungen sind `.mp4` oder `.m4a`.
- **Ogg:** Ein Open-Source-Standard, die Dateierendung lautet `.ogg` oder `.oga`.
- **WebM:** Ein Open-Source-Format von Google, die Dateierendung lautet `.webm`.
- **WAVE:** Ein populäres Format, das meist PCM-Daten enthält, Endung `.wav`.
- **FLAC:** Ein Containerformat für den gleichnamigen Audio-Codec, mit Endung `.flac`.

In diesen Containern können sich die folgenden Codecs bzw. Formate wiederfinden:

- **MP3:** MPEG-1 Audio Layer 3 ist der wohl am weitesten verbreitete verlustbehaftete Audio-Codec.
- **AAC:** Advanced Audio Coding ist der Standardcodec für Audio auf Apple-Systemen. Der Codec ist ebenfalls lizenzpflichtig.
- **Vorbis:** Ein lizenzfreier Open-Source-Codec, meist im Zusammenhang mit dem Ogg-Containerformat verwendet.
- **FLAC:** Ein freier, verlustfreier Audio-Codec.
- **PCM:** Unkomprimierte Audiodaten im Pulse-Code-Modulationsverfahren.

Tabelle 12.2 stellt dar, welche Kombinationen möglich sind und wie sie in den einzelnen Browsern unterstützt werden. AAC in Firefox ist nur möglich, wenn die Codecs im

Tabelle 12.2: Audio-Containerformate mit Codecs und Browserunterstützung. C144: Chrome 144, FF147: Firefox 147, E144: Edge 144, O125: Opera 125, S26.2: Safari 26.2.

<i>Format</i>	<i>C144</i>	<i>FF147</i>	<i>E144</i>	<i>O125</i>	<i>S26.2</i>
WAVE: PCM	•	•	•	•	•
WebM: Vorbis	•	•	•	•	•
Ogg: Vorbis	•	•	•	•	•
MP3	•	•	•	•	•
MP4: MP3	•	•	•	•	•
MP4: AAC	•	◦	•	•	•
FLAC	•	•	•	•	•
Ogg: FLAC	•	•	•	•	•



Abbildung 12.9: Die Audio-Kontrollelemente in Google Chrome.

System bereits installiert sind. Alle anderen Kombinationen funktionieren in modernen Browsern.

12.4.2 Audio Markup

Wie beim Video-Element kann im einfachsten Fall einfach eine Datei als Quelle im `src`-Attribut des `<audio>`-Elements angegeben werden. Das Attribut `controls` ist nötig, um überhaupt die Kontrollelemente zur Wiedergabe zu sehen.

```
1 <audio src="../../assets/miaow-bubble.mp3" controls>Ihr Browser unterstützt Audio
   leider nicht!</audio>
```

Wiederum sollte auf Nummer sicher gegangen werden und mit Hilfe der `<source>`-Elemente innerhalb von `<audio>` mehrere Formate angeboten werden, damit eine durchgängige Browserunterstützung gegeben ist. Abbildung 12.9 zeigt die reinen Audio-Kontrollelemente mit einer bzw. auch mehreren Quelldateien.

```
1 <audio controls>
2   <source src="../../assets/miaow-bubble.flac" type="audio/flac">
3   <source src="../../assets/miaow-bubble.mp3" type="audio/mpeg">
4   <source src="../../assets/miaow-bubble.ogg" type="audio/ogg">
5 </audio>
```

12.4.3 Kontrollelemente

Die API für Audio ist identisch zur der von Video. Auch hier können mit eigenen Elementen wie Buttons und dem entsprechenden JavaScript-Code Audiodateien abgespielt, gestoppt und etwa die Lautstärke verändert werden.

Die Methoden `play()` und `pause()` erlauben das Abspielen bzw. Pausieren, die Eigenschaft `volume` enthält die Lautstärke. Abschnitt 12.3.3 kann als Referenz für die Implementierung herangezogen werden.

12.5 Geolocation

Geolocation ist die Möglichkeit abzufragen, wo in der Welt sich die Benutzer*innen, die gerade auf der Website surfen, befinden. Die Art und Weise, wie die Position dabei bestimmt wird, hängt viel auch vom verwendeten Endgerät ab. Smartphones und Tablets bieten zumeist integrierte GPS-Module, wodurch eine Positionsbestimmung (im Freien) sehr genau möglich ist. Aber auch ohne dieses Hilfsmittel kann die Position ermittelt werden. Dabei werden etwa die IP-Adresse, MAC-Adresse oder verfügbare Drahtlosnetzwerke herangezogen.

Mit Hilfe der Geolocation API [9] kann diese Position ermittelt und für eigene Webanwendungen verwendet werden. Wichtig dabei ist, dass die Abfrage der Position niemals ohne Zustimmung der Benutzer*innen erfolgen kann und darf. Daher zeigt der Browser Anwender*innen zunächst eine Abfrage, ob diese ihre Positionsdaten mit der aktuellen Website teilen möchten. Stimmt der*die Benutzer*in nicht zu, steht diese Information nicht zur Verfügung.

12.5.1 Abfragen der aktuellen Position

Die Abfrage der Position geschieht über das `navigator`-Objekt bzw. dessen Unterobjekt `geolocation`, das die Methode `getCurrentPosition()` zur Verfügung stellt. Diese erwartet als Parameter eine sogenannte Callback-Funktion, also eine Funktion, die automatisch aufgerufen wird, sobald die Ergebnisse der Positionsabfrage vorliegen. Dies kann entweder eine selbst geschriebene, eigene Funktion sein, oder auch eine anonyme Funktion. Wichtig ist nur, dass immer der Parameter `position` vorhanden sein muss, denn er enthält die aktuelle Position.

Ein Aufruf mit einer eigenen Funktion namens `showMap()` könnte also so aussehen:

```
1 navigator.geolocation.getCurrentPosition(showMap);
2
3 function showMap(position) {
4   const lat = position.coords.latitude;
5   const long = position.coords.longitude;
6   // Zeige eine Karte
7 }
```

Das Gleiche kann ebenso mit einer anonymen Funktion so erreicht werden:

```
1 navigator.geolocation.getCurrentPosition(function(position) {
2   const lat = position.coords.latitude;
3   const long = position.coords.longitude;
4   // Zeige eine Karte
5 });
```

Wie ersichtlich ist, muss die Verarbeitung der Position innerhalb der Callback-Funktion (egal ob normal oder anonym) erfolgen, denn `getCurrentPosition()` liefert nicht wie eine normale Methode einen Wert zurück, sondern ruft die Callback-Funktion auf.

12.5.2 Anzeigen einer Karte

Um eine Karte mit der aktuellen Position anzeigen zu können, wird *Leaflet*, eine JavaScript-Bibliothek zum Anzeigen von Karten verwendet. Die Kartenteile kommen von OpenStreetMap. Alternativ kann auch die Google Maps JavaScript API⁶ verwendet werden. Diese benötigt jedoch einen API-Schlüssel, welcher nur gegen Hinterlegen einer Kreditkarte erhältlich ist und bei zu viele Aufrufen Kosten erzeugt.

Leaflet wird am einfachsten online, von einem CDN (Content Delivery Network) im `<head>` mit `defer` eingebunden. Neben der JavaScript-Datei wird auch noch eine CSS-Datei benötigt, die für die Formatierung der Kartenelemente sorgt:

```
1 <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"
  integrity="sha256-p4NxAoJBhIIN+hmNhrzRCf9tD/miZyoHS5obTRR9BMY=" crossorigin=""/>
2 <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js" integrity="sha256-20
  nQCchB9co0qIjJZRGuk2/Z9VM+kNiyxNV1lvTlZBo=" crossorigin="" defer></script>
```

Um nun die Karte anzuzeigen, wird im `<body>` ein `<div>`-Element mit der id `map` angelegt:

```
1 <div id="map"></div>
```

Der JavaScript-Code zum Anzeigen der Karte wird in eine Datei `map.js` ausgelagert, die nach Leaflet ebenfalls mit `defer` eingebunden wird. Die Funktion `initialize()` erzeugt die Karte im `<div>` mit der ID `map` und fügt einen Tile-Layer (die sichtbaren Kacheln der Karte) hinzu:

```
1 function initialize() {
2   const map = L.map("map").setView([47.61, 13.783], 8);
3
4   // Add a tile layer: we use OpenStreetMap here
5   L.tileLayer("https://tile.openstreetmap.org/{z}/{x}/{y}.png", {
6     maxZoom: 19,
7     attribution: '&copy; <a href="http://www.openstreetmap.org/copyright">
      OpenStreetMap</a>'
8   }).addTo(map);
9
10  // Geolocation Code
11 }
```

Mit `L.map()` wird die Karte erzeugt, die Funktion `setView()` erhält das Zentrum der Karte und eine initiale Zoomstufe. Der Tile-Layer stammt von OpenStreetMap und wird ebenfalls der Karte hinzugefügt.

Nun fehlt noch die aktuelle Position, auf der die Karte zentriert sein soll. Hier kommt Geolocation zum Einsatz. Zunächst wird überprüft, ob Geolocation verfügbar ist, dann die Position wie oben gezeigt ermittelt und schließlich auf der Karte mit der Methode `setView()` gesetzt. Weiters wird ein Text ins einen Marker geschrieben und der Zoomlevel angepasst.

Sollte der*die Nutzer*in die Anfrage zur Positionsbestimmung ablehnen oder der Browser Geolocation nicht unterstützen, wird die Funktion `handleLocationError` aufgerufen. Diese zeigt im Marker die entsprechende Fehlermeldung an.

```
1 function initialize() {
2   // Karten Initialisierung
```

⁶<https://developers.google.com/maps/documentation/javascript/>

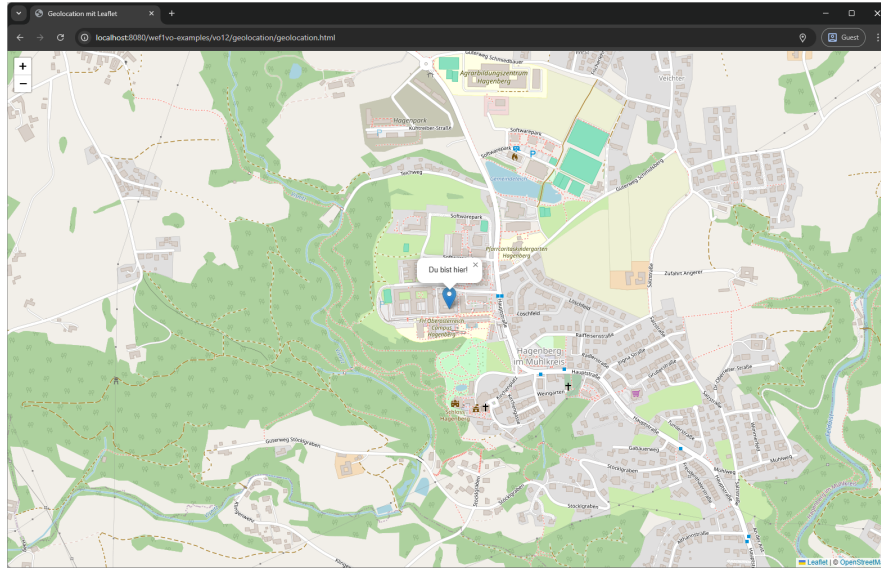


Abbildung 12.10: Der Browser zeigt (in der FH) eine Vollbildkarte mit Hagenberg im Zentrum an.

```

3
4  if (navigator.geolocation) {
5      navigator.geolocation.getCurrentPosition(position => {
6          const lat = position.coords.latitude;
7          const lng = position.coords.longitude;
8          map.setView([lat, lng], 16);
9
10         L.marker([lat, lng]).addTo(map)
11             .bindPopup("Du bist hier!")
12             .openPopup();
13     }, () => {
14         handleLocationError(true, map);
15     });
16 } else {
17     // Browser unterstützt keine Geolocation
18     handleLocationError(false, map);
19 }
20 }

```

Dieses Beispiel erzeugt eine Vollbildkarte mit der aktuellen Position und einer Nachricht im Zentrum. Das Ergebnis ist in Abbildung 12.10 ersichtlich.

12.6 Local Storage

Die Fähigkeit, Daten lokal speichern zu können, war immer ein typischer Vorteil von nativen Client-Applikationen gegenüber Web-Applikationen. Für Websites gab es nur Cookies zum Speichern von lokalen Daten, doch diese waren nie für diesen Zweck gedacht. Zunächst können sie nur 4 KB an Daten speichern, weiters werden Sie zwingend bei jedem HTTP-Request übertragen, was nicht sehr effizient ist.

Hier schafft die HTML *Web Storage API* [5, Abschnitt 12] Abhilfe. Sie stellt einen lokalen Speichermechanismus zur Verfügung, der die Speicherung beliebiger Schlüssel-Wert-Paare erlaubt. Diese Werte bleiben selbst dann gespeichert, wenn von der aktuellen Seite weg navigiert oder das Browserfenster geschlossen wird.

Der Zugriff auf diesen lokalen Speicher ist über das `localStorage`-Objekt möglich. Es ist ein Unterobjekt von `window`, also immer global verfügbar. Es besitzt folgende wichtige Methoden:

- `setItem(key, value)`: Speichert ein neues Schlüssel/Wert Paar mit dem Schlüssel `key` und dem Wert `value`.
- `getItem(key)`: Liefert den Wert des Local Storage Eintrags mit dem Schlüssel `key` zurück.
- `removeItem(key)`: Löscht den Eintrag mit dem Schlüssel `key` aus dem Local Storage.

Im folgenden Beispiel soll eine simple Post-It Applikation erzeugt werden. Als Post-It dient ein `<section>`-Element mit 200×200 Pixeln. Das Element enthält das Attribut `contenteditable` mit dem Wert `true`. Dieses HTML-Attribut ermöglicht das Editieren von Text direkt an Ort und Stelle. Der einzige Nachteil: Wird die Seite neu geladen, sind alle Änderungen wieder verschwunden. Hier schafft *Local Storage* Abhilfe. Zunächst aber der HTML-Code für das Post-It (die Formatierung erfolgt entsprechend in CSS). Das Element erhält weiters eine `id` zum leichteren Auffinden in JavaScript.

```
1 <section id="sticky" contenteditable="true"></section>
```

Im JavaScript-Code wird zunächst ein Standardtext definiert, dann das Post-It-Element gesucht und danach wird mittels der `getItem()`-Methode abgefragt, ob im Local Storage bereits ein Inhalt vorhanden ist. Dies wäre der Fall, falls mit dem Post-It bereits interagiert wurde. In diesem Fall würde der bestehende Inhalt als HTML-Inhalt eingefügt. Beim ersten Aufruf der Seite wird diese Abfrage fehlschlagen und es wird ein neuer Standardinhalt in das `<section>`-Element geschrieben. Dies wird mit der Eigenschaft `innerHTML` gelöst. Diese ersetzt den Inhalt eines HTML-Elements durch den gegebenen String. Kommen darin andere HTML-Elemente vor, so werden diese ebenfalls entsprechend verschachtelt.

Vorsicht: Wenngleich die Eigenschaft `innerHTML` praktisch ist, so kann sie ein Sicherheitsrisiko für sogenanntes Cross-Site-Scripting⁷ darstellen.

Sodann wird auf das Element ein `blur`-Event-Handler gelegt. Der `blur`-Event wird ausgelöst, wenn neben das `<section>`-Element geklickt wird. Dies passiert immer dann, wenn Benutzer*innen aufhören wollen, das Post-It zu editieren, da sie zum Schreiben einfach hinein klicken müssen. Sind sie fertig, wird überprüft, ob der Inhalt leer ist. Dies wird mit der Eigenschaft `textContent` überprüft. Diese funktioniert so ähnlich wie `innerHTML`, beschränkt sich jedoch rein auf Text. Enthaltene HTML-Elemente werden ignoriert. Dies ist für diese Abfrage vorteilhaft, da die Browser leere Elemente nicht komplett leer zurücklassen. So bleibt meist etwa ein `<br>`-Element zurück, wodurch bei der Verwendung von `innerHTML` das Element nicht mehr als leer erkannt würde.

Ist das Element leer (d. h. enthält es keinerlei Text) oder nur den Standardtext, wird der Local Storage-Eintrag gelöscht, da der*die Benutzer*in offensichtlich Inhalt

⁷<https://developer.mozilla.org/en-US/docs/Web/API/Element/innerHTML>

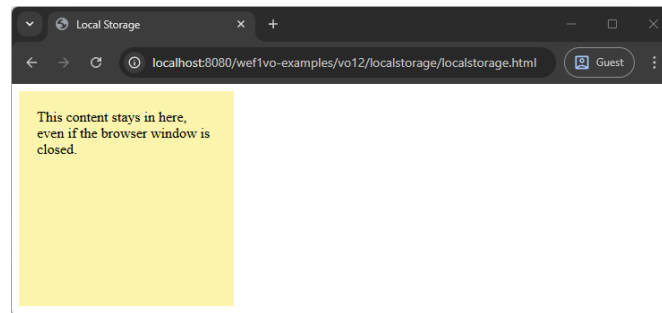


Abbildung 12.11: Der Text im Post-It kann einfach durch Anklicken geändert werden. Er bleibt lokal im Browser gespeichert und ist beim nächsten Aufruf der Seite wieder vorhanden.

entfernt hat. Danach wird der Standardtext wieder ins Element eingefügt, damit das Post-It nicht leer bleibt. Ist aber nach dem `blur`-Event noch Text vorhanden, wird er in den Local Storage geschrieben. Hier kommt wiederum der Inhalt mittels `innerHTML` zum Zug, da so auch mögliche Zeilenumbrüche mitgespeichert werden.

Wird nun die Seite neu geladen, oder nach dem Schließen des Browserfensters wieder geöffnet, steht der Inhalt nach wie vor im Post-It.

Im Folgenden der Code, das Ergebnis ist in Abbildung 12.11 zu sehen:

```
1 const defaultText = "Schreib auf mich!";
2 const note = document.getElementById("sticky");
3 const storedValue = localStorage.getItem("content");
4
5 if (storedValue) {
6   note.innerHTML = storedValue;
7 } else {
8   note.innerHTML = defaultText;
9 }
10
11 note.addEventListener("blur", (event) => {
12   const plainText = event.target.textContent;
13   if (plainText.trim() === "" || plainText === defaultText) {
14     localStorage.removeItem("content");
15     event.target.innerHTML = defaultText;
16   } else {
17     localStorage.setItem("content", event.target.innerHTML);
18   }
19 });
```

12.7 Drag & Drop

Um *Drag & Drop* zu realisieren, also ein Element mit der Maus anzuklicken und mit gedrückter Maustaste woanders hinzubewegen, musste lange auf JavaScript-Bibliotheken wie jQuery zurückgegriffen werden. Dank der HTML Drag & Drop API [5, Abschnitt 6.11] ist dies auch komplett ohne Bibliothek möglich. Dafür sind grundsätzlich vier Schritte notwendig:

1. Elemente ziehbar machen.
2. Mit dem Ziehen von Elementen starten.
3. Elemente zum Fallenlassen auswählen.
4. Operationen beim Fallenlassen ausführen.

Diese vier Schritte werden im Folgenden an einem Beispiel erklärt, in dem ein Bild zwischen drei `<div>`-Elementen verschoben werden kann.

12.7.1 Elemente ziehbar machen

Zunächst müssen Elemente bestimmt werden, mit denen Drag & Drop überhaupt möglich ist. Standardmäßig gehören dazu Bilder oder auch Links, mit Hilfe des Attributs `draggable` kann jedoch für jedes Element festgelegt werden, ob es ziehbar ist oder nicht. Wird der Wert des Attributs auf `true` gesetzt, ist dies der Fall, mit `false` kann das Ziehen unterbunden werden.

Im folgenden Beispiel existieren drei `<div>`-Elemente, die in weiterer Folge als Ziel für Drag & Drop-Operationen dienen werden. Im ersten dieser Elemente liegt ein Bild, das mit Hilfe des `draggable`-Attributs explizit als ziehbar gekennzeichnet wurde. Es besitzt weiters die ID `dragImage` mit dem es während der Operation identifiziert werden kann.

```
1 <div class="dropzone">
2   
3 </div>
4
5 <div class="dropzone"></div>
6
7 <div class="dropzone"></div>
```

12.7.2 Mit dem Ziehen von Elementen starten

Nachdem nun die ziehbaren Elemente festgelegt wurden, muss definiert werden, was beim Start eines Ziehvorgangs passiert. Dazu wird auf den `dragstart`-Event reagiert, der genau auf diesen Vorgang hin ausgelöst wird. Dazu wird (das Script liegt am Ende des `<body>`-Elements) zunächst das ziehbare Element ausgewählt und auf diesem ein `dragstart`-Listener angebracht. Innerhalb dieses Listeners werden dem Event-Objekt Daten mitgegeben, die während des Drag & Drop Vorgangs relevant sind. Diese Daten werden im Objekt `dataTransfer` gespeichert, das ein Unterobjekt des `event`-Objekts ist. Mit dessen Methode `setData()` können Daten hinzugefügt werden. Der erste Parameter ist der Typ (etwa einfacher Text), der zweite Parameter sind die Daten selbst.

Im Beispiel wird beim Start des Ziehens die ID des gezogenen Elements (`dragImage`) beim `event`-Objekt (enthält alle Daten über das Element, bei dem der Event aufgetreten ist) gespeichert. Sie kann über die Eigenschaft `target` ausgelesen werden.

```
1 const dragImage = document.getElementById("dragImage");
2
3 dragImage.addEventListener("dragstart", event => {
4   event.dataTransfer.setData("text", event.currentTarget.id);
5 });
```

12.7.3 Elemente zum Fallenlassen auswählen

Im nächsten Schritt muss festgelegt werden, auf welchen Elementen das gezogene Element überhaupt fallengelassen werden kann. Dafür steht der **dragover**-Event zur Verfügung. Um ein Fallenlassen auf ein Element zu erlauben, muss ein Listener auf diesem Event angebracht werden und dort mittels **event.preventDefault()** das Standardverhalten dieses Events unterbunden werden. Dieses verbietet nämlich das Fallenlassen, wird das Standardverhalten also deaktiviert, ist ein Fallenlassen möglich.

Im Beispiel werden zunächst die drei **<div>**-Elemente mit der Klasse **dropzone** gesucht. In einer **ForEach**-Schleife wird über die gefundenen Elemente iteriert und bei jedem ein Event-Listener mit der entsprechenden Anweisung registriert.

```
1 const dropZones = document.querySelectorAll(".dropzone");
2
3 dropZones.forEach(zone => {
4   zone.addEventListener("dragover", event => {
5     event.preventDefault();
6   });
7   ...
8 });
```

12.7.4 Operationen beim Fallenlassen ausführen

Wird schließlich über einem Element, bei dem Fallenlassen erlaubt wurde, die Maustaste losgelassen, wird der **drop**-Event ausgelöst. Im entsprechenden Event-Listener kann nun also festgelegt werden, was in diesem Moment passiert. In der Regel werden hier die Daten aus dem **dataTransfer**-Objekt entnommen und verarbeitet.

Im Beispiel passiert genau dies. Zunächst wird die ID des gezogenen Elements mit der Methode **getData()** entnommen und gespeichert. Dann wird im DOM das Element mit dieser ID (das Bild) gesucht und ebenfalls mittels DOM-Methode in das Element gehängt, auf dem der **drop**-Event aufgetreten ist. **event.preventDefault()** sollte auch hier aufgerufen werden, um zu vermeiden, dass beim gezogenen Objekt das Standardverhalten ausgelöst wird. Wäre das Element etwa ein Link, würde diesem gefolgt werden (was meist nicht gewünscht ist).

```
1 dropZones.forEach(zone => {
2   ...
3   zone.addEventListener("drop", event => {
4     event.preventDefault();
5     const data = event.dataTransfer.getData("text");
6     const draggedElement = document.getElementById(data);
7     event.currentTarget.appendChild(draggedElement);
8   });
9 });
```

Abbildung 12.12 stellt den Drag & Drop-Vorgang im Beispiel schematisch dar.

12.7.5 Weitere Events und Anmerkungen

Neben den gezeigten Events werden noch eine Reihe weiterer beim Drag & Drop-Vorgang ausgelöst. Dazu zählen etwa **dragenter** (wird beim Ziehen über ein Element



Abbildung 12.12: Das HTML5-Logo wird vom ersten Element zum zweiten gezogen und kann dort im Anschluss abgelegt werden.

ausgelöst), `dragleave` (wird beim Verlassen eines Elements ausgelöst) und `dragend` (wird beim Beenden des gesamten Vorgangs ausgelöst).

Ebenso kann im Prozess bestimmt werden, wie sich der Mauscursor verändert, was hilfreich ist um anzudeuten, wo Elemente fallengelassen werden können und wo nicht.

Weiters sei angemerkt, dass auch Sonderfälle berücksichtigt werden sollten, etwa was passiert, wenn die Maus das Browserfenster verlässt. Um große Datenmengen mit Drag & Drop zu verarbeiten sind zusätzliche APIs wie etwa die File API [8] notwendig.

12.8 Weitere Technologien

Die hier vorgestellten Technologien sind natürlich längst nicht alles, was es an Web APIs gibt. Folgende Dinge würden den Rahmen dieser Vorlesung sprengen, sie seien aber der Vollständigkeit halber erwähnt.

12.8.1 Web Sockets

HTTP ist ein zustandsloses Protokoll. Nach der Übermittlung von den Daten weiß das Protokoll nichts mehr von dieser Übermittlung. Permanent offene Verbindungen sind somit nicht möglich. *Web Sockets* [6] schaffen hierbei Abhilfe. Mit ihnen ist es möglich, Dinge quasi in Echtzeit zu erledigen, denn sie erhalten eine beständige Verbindung zum Server aufrecht. Ein Beispiel wäre ein webbasierter Chat. Beim Laden der Seite verbindet sich der Chat zum Chatserver. Solange die Verbindung offen ist, können Nachrichten verschickt und empfangen werden.

12.8.2 Indexed Database

Die vorgestellte Local Storage Implementierung ist eine schnelle Art und Weise, Daten lokal zu speichern. Komplexere Datenmodelle sind jedoch kaum damit abzubilden. Mit Hilfe der *Indexed Database API* [10] wurde eine Schnittstelle geschaffen, mit der eine indizierte Datenbank verwaltet werden kann. Derzeit implementieren alle wichtigen Browser diese Spezifikation komplett.

12.8.3 Web Workers

JavaScript ist eine sogenannte „single-threaded language“, d. h. sie kann immer nur einen Codestrang ausführen. Mehrere (pseudo) parallele Abarbeitungen sind nicht möglich. *Web Workers* [5, Abschnitt 10] heben diese Einschränkung auf. Mit ihnen ist es möglich, im Hintergrund parallel Code auszuführen, ohne dass die Website im Vordergrund dadurch beeinträchtigt wird („hängen bleibt“).

12.8.4 Service Workers

Um Webapplikationen zu realisieren, die auch offline funktionieren, wurde das Konzept der *Service Workers* [11] erdacht. Sie agieren als Netzwerk Proxy und ermöglichen volle Kontrolle, wie Kommunikation über das Netzwerk gehandhabt wird. Eine E-Mail-Applikation kann also etwa auch offline funktionieren und erst dann Daten senden, wenn wieder eine Verbindung besteht.

12.9 Weiterführende Literatur

Literatur zu diesem Thema ist derzeit noch spärlich verfügbar. Aufgrund der aktuellen Thematik ist sie auch meist bereits veraltet. Es gibt dennoch einige Werke, die empfehlenswert sind – für alle dort nicht abgedeckten Dinge muss auf die Dokumentation des jeweiligen Entwurfs vom W3C bzw. der WHATWG zurück gegriffen werden.

Eine Übersicht zu den gezeigten (und einigen weiteren) APIs bietet [7], etwas weiter ins Detail bei einigen APIs geht [1].

Die meisten Bücher zu Spezialthemen wie Canvas oder Geolocation sind schon über 10 Jahre alt und werden ohnehin durch generelle HTML- oder JavaScript-Literatur abgedeckt. Hier sei auf Codebeispiele und Lernplattformen im Web verwiesen.

Quellenverzeichnis

- [1] Philip Ackermann. *JavaScript. Das umfassende Handbuch*. 4. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Nov. 2025 (siehe S. 12-23).
- [2] Khronos Group. *WebGL 2 Specification*. Version 2.0.0. 11. Apr. 2017. URL: <https://registry.khronos.org/webgl/specs/2.0/> (siehe S. 12-3).
- [3] Khronos Group. *WebGL Specification*. Version 1.0.3. 27. Okt. 2014. URL: <https://registry.khronos.org/webgl/specs/1.0/> (siehe S. 12-3).
- [4] Mark Pilgrim. *HTML5. Up and Running*. Sebastopol, USA: O'Reilly, 6. Aug. 2010 (siehe S. 12-5).
- [5] Web Hypertext Application Technology Working Group. *HTML*. Living Standard. 23. Jan. 2026. URL: <https://html.spec.whatwg.org/multipage/> (siehe S. 12-1, 12-2, 12-3, 12-8, 12-18, 12-19, 12-23).
- [6] Web Hypertext Application Technology Working Group. *WebSockets*. Living Standard. 12. Aug. 2025. URL: <https://websockets.spec.whatwg.org/> (siehe S. 12-22).
- [7] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 12-23).
- [8] World Wide Web Consortium. *File API*. W3C Working Draft. 3. Dez. 2025. URL: <https://www.w3.org/TR/FileAPI/> (siehe S. 12-22).
- [9] World Wide Web Consortium. *Geolocation*. W3C Recommendation. 29. Okt. 2025. URL: <https://www.w3.org/TR/geolocation/> (siehe S. 12-1, 12-15).
- [10] World Wide Web Consortium. *Indexed Database API 3.0*. W3C Working Draft. 13. Aug. 2025. URL: <https://www.w3.org/TR/IndexedDB/> (siehe S. 12-22).
- [11] World Wide Web Consortium. *Service Workers Nightly*. W3C Candidate Recommendation Draft. 26. Jan. 2026. URL: <https://www.w3.org/TR/service-workers/> (siehe S. 12-23).